

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA0615	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	10% of CIA	Total Marks:	100 (2 Questions × 50 Marks)
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	SOUNDARIYA M	Reg. No.:	192524342
Section / Batch:	B.TECH AI&DS	Date:	24-08-2026

Instructions to Students

- Answer both questions carefully. Each question carries 50 marks. Total: 100 marks.
- Analyze the given questions thoroughly before answering.
- Compare multiple aspects such as methods, results, advantages, and limitations clearly.
- Critically evaluate the strengths, weaknesses, and implications with proper justification.
- Present meaningful insights, interpretations, and well-supported conclusions from the comparisons.

Question Type	Description	Questions	Total Marks
Comparative Analysis	Focus on Comparison of different Algorithms using dynamic programming and greedy strategies.	Q1 – Q2	100

SECTION A – Comparative Analysis

- Define the Word Break and Longest Palindromic Subsequence problems with suitable examples.
- Write the DP algorithm or pseudocode.
- Prepare the DP table/state representation.
- Develop the recurrence relation
- Implement the Longest Palindromic Subsequence using Dynamic Programming (Saveetha Compiler)
- Analyze time complexity.
- Analyze space complexity.
- Compare the two DP formulations using a table.
- Identify similarities between the two problems.
- Identify differences in state definition and transitions.
- Compare the DP state, transition, implementation, and complexity of both problems and provide a conclusion.

1. Knapsack Comparison (DP vs Greedy)

Given items with weights [10, 20, 30] and values [60, 100, 120] with capacity 50, solve using both 0/1 Knapsack (Dynamic Programming) and Fractional Knapsack (Greedy). Compare the solutions obtained and analyze differences in optimality and approach. Evaluate the strengths and limitations of both methods. Discuss when greedy provides optimal results and when DP is necessary. Generate insights on method selection

Given:

Weights	=	{10,	20,	30}
Values	=	{60,	100,	120}

Knapsack Capacity = 50

1. Problem Definition with Suitable Example

The **Knapsack Problem** is an optimization problem in which a set of items is given with different weights and values. The objective is to select items so that the total weight does not exceed the knapsack capacity while maximizing the total value.

In **0/1 Knapsack**, each item can either be completely selected or completely rejected. Fractions of items are not allowed. Dynamic Programming is used to find the optimal solution.

In **Fractional Knapsack**, an item can be divided into fractions. A Greedy approach is used by selecting items according to their value-to-weight ratio.

For the given problem:

Item	Weight	Value	Value/Weight
------	--------	-------	--------------

Item 1	10	60	6
--------	----	----	---

Item 2	20	100	5
--------	----	-----	---

Item 3	30	120	4
--------	----	-----	---

Capacity = **50**

For 0/1 Knapsack, the best selection is Item 2 and Item 3:

For Fractional Knapsack, Item 1 and Item 2 are selected completely and of Item 3 is selected:

2. Algorithm / Pseudocode

Dynamic Programming - 0/1 Knapsack

1. Initialize the DP table with zero.
2. Consider each item one by one.
3. For every capacity, check whether the current item can be included.

4. If included, compare the value obtained by including and excluding the item.
5. Store the maximum value.
6. The final DP value gives the optimal solution.

Pseudocode:

Initialize $dp[capacity] = 0$

For each item:

For capacity from maximum to item weight:

```
dp[capacity] =
max(dp[capacity],
value + dp[capacity - weight])
```

Return $dp[capacity]$

Greedy - Fractional Knapsack

1. Calculate value/weight ratio for every item.
2. Sort the items in decreasing order of ratio.
3. Select the item with the highest ratio first.
4. If the complete item fits, select it.
5. Otherwise, select the required fraction of the item.
6. Continue until the capacity becomes zero.

3. DP Table / State Representation

Let:

represent the maximum value obtained using the first i items with capacity w .

A compact representation of the DP states is:

Items	10	20	30	40	50
Item 1	60	60	60	60	60
Item 1, 2	60	100	160	160	160
Item 1, 2, 3	60	100	160	180	220

Therefore, for capacity 50:

The selected items are **Item 2 and Item 3**.

4. Recurrence Relation

For 0/1 Knapsack, if the item weight is less than or equal to the current capacity:

If the item cannot fit:

where:

- = current item
- = current capacity
- = weight of item
- = value of item

For Fractional Knapsack, the greedy decision is based on:

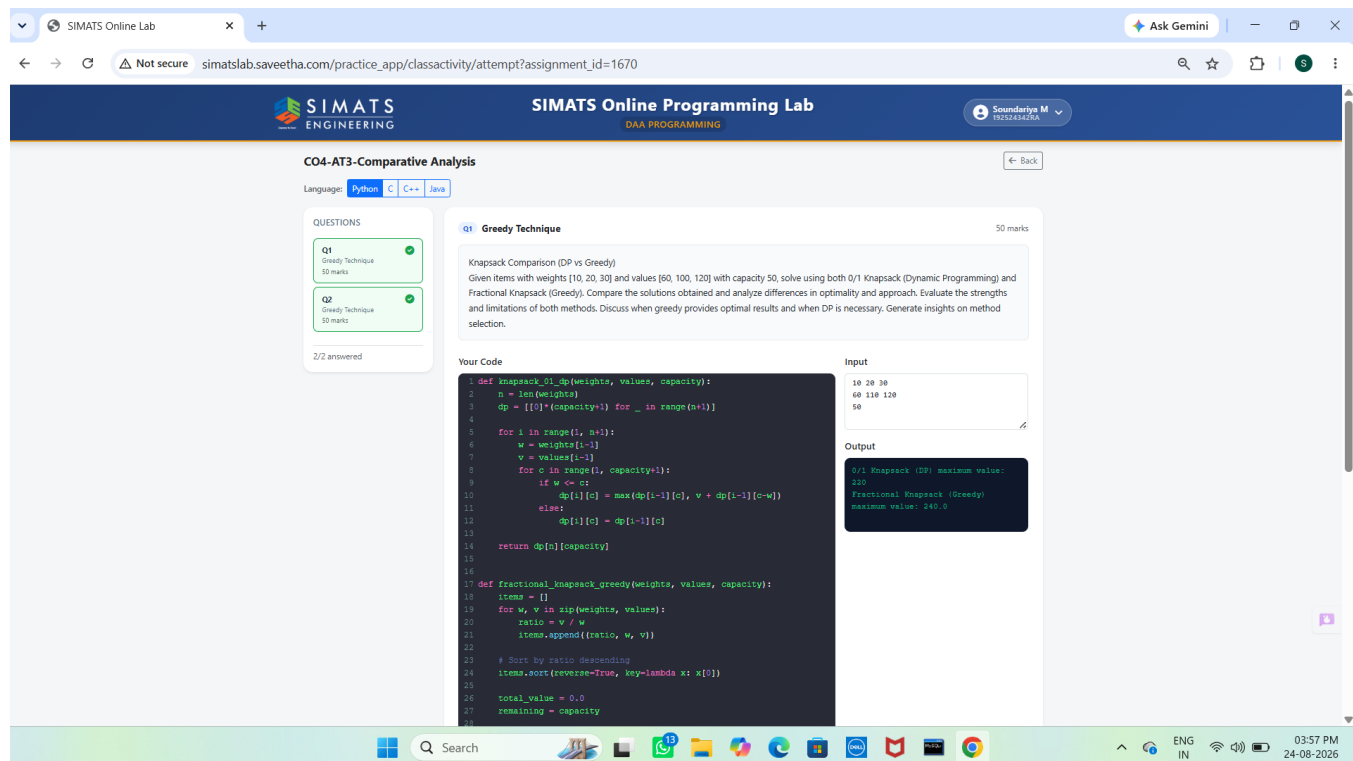
The items are selected in decreasing order of this ratio.

For the given items:

Therefore, the selection order is:

Item 1 → Item 2 → Item 3

5. Implementation



The screenshot shows the SIMATS Online Programming Lab interface. The top navigation bar includes the SIMATS Engineering logo, the lab name, and a user profile. The main content area displays a question titled "CO4-AT3-Comparative Analysis" with a sub-question "Q1 Greedy Technique" worth 50 marks. The question text describes a Knapsack Comparison problem (DP vs Greedy) with given items and capacity, asking to solve using both 0/1 Knapsack (Dynamic Programming) and Fractional Knapsack (Greedy) and compare the solutions. The interface includes a "Your Code" section with a Python code editor, an "Input" section with a text area, and an "Output" section showing the results of the code execution. The code defines two functions: `knapsack_dp` for Dynamic Programming and `fractional_knapsack_greedy` for the Greedy approach. The input values are 10, 20, 30 for weights and 60, 100, 120 for values, with a capacity of 50. The output shows the maximum values for both methods: 120 for DP and 240.0 for Greedy.

```
1 def knapsack_dp(weights, values, capacity):
2     n = len(weights)
3     dp = [[0]*(capacity+1) for _ in range(n+1)]
4
5     for i in range(1, n+1):
6         w = weights[i-1]
7         v = values[i-1]
8         for c in range(1, capacity+1):
9             if w <= c:
10                dp[i][c] = max(dp[i-1][c], v + dp[i-1][c-w])
11            else:
12                dp[i][c] = dp[i-1][c]
13
14     return dp[n][capacity]
15
16
17 def fractional_knapsack_greedy(weights, values, capacity):
18     items = []
19     for w, v in zip(weights, values):
20         ratio = v / w
21         items.append((ratio, w, v))
22
23     # Sort by ratio descending
24     items.sort(reverse=True, key=lambda x: x[0])
25     total_value = 0.0
26     remaining = capacity
27
28     for ratio, w, v in items:
29         if remaining < w:
30             total_value += ratio * remaining
31             remaining = 0
32             break
33         remaining -= w
34         total_value += v
```

Input:

```
10 20 30
60 100 120
50
```

Output:

```
0/1 Knapsack (DP) maximum value:
120
Fractional Knapsack (Greedy):
Maximum value: 240.0
```

6. Time Complexity Analysis

For 0/1 Knapsack using Dynamic Programming, there are n items and capacity C .

Therefore:

For the given problem:

For Fractional Knapsack using Greedy, the items need to be sorted according to their value/weight ratio.

Sorting takes:

Therefore, Greedy is computationally faster than DP when the capacity is large.

7. Space Complexity Analysis

For the 0/1 Knapsack DP implementation using a one-dimensional array:

For Fractional Knapsack, storing the items and their ratios requires:

Thus, the DP approach requires space based on the knapsack capacity, while the Greedy approach requires space based mainly on the number of items.

8. Comparison of the Two Approaches

Parameter	0/1 Knapsack - DP	Fractional Knapsack - Greedy
Technique	Dynamic Programming	Greedy
Item division	Not allowed	Allowed
Selection	Complete item	Complete or fractional
Decision	Considers combinations	Highest ratio first
Result for given problem	220	240
Time Complexity	$O(nC)$	$O(n \log n)$
Space Complexity	$O(C)$	$O(n)$
Optimality	Optimal for 0/1 Knapsack	Optimal for Fractional Knapsack

The values **220** and **240** should not be treated as a direct competition, because the two methods solve different versions of the Knapsack problem.

9. Similarities Between the Two Approaches

1. Both approaches solve Knapsack optimization problems.
 2. Both attempt to maximize the total value.
 3. Both consider the weight and value of each item.
 4. Both have a fixed knapsack capacity.
 5. Both produce an optimal solution for the version of the problem for which they are designed.
 6. Both can be implemented using Python.
-

10. Differences in State Definition and Transitions

In **Dynamic Programming**, the state represents the maximum value possible for a particular number of items and capacity.

The transition considers **two choices**:

- Include the item.
- Exclude the item.

Therefore, DP evaluates different combinations.

In **Greedy**, there is no DP state table. The algorithm calculates the value/weight ratio and selects the item with the highest ratio available.

For the given problem:

Hence, the Greedy transition is based on:

The main difference is that **DP considers combinations**, while **Greedy makes the best local choice at each step**.

11. Comparison of State, Transition, Implementation and Complexity

Aspect	Dynamic Programming	Greedy
State	dp[i][w] represents maximum value	No DP state
Transition	Include or exclude item	Select highest value/weight ratio
Implementation	DP table/array	Sorting + selection
Decision type	Global optimization through subproblems	Local optimal choice
Time	$O(nC)$	$O(n \log n)$
Space	$O(C)$	$O(n)$
Best suited for	0/1 Knapsack	Fractional Knapsack
Main limitation	Higher computation for large capacity	Not guaranteed for 0/1 Knapsack

For the given input, DP obtains **220** because only complete items can be selected. Greedy obtains **240** because it is allowed to take a fraction of Item 3.

12. Conclusion

The comparison shows that Dynamic Programming is appropriate for the 0/1 Knapsack problem, where items cannot be divided. It evaluates different combinations and guarantees the optimal solution.

The Greedy approach is appropriate for Fractional Knapsack, where items can be divided. Selecting items according to the highest value/weight ratio produces an optimal solution.

For the given input:

Therefore, algorithm selection depends on the nature and constraints of the problem. DP is necessary when complete-item selection must be optimized, while Greedy is preferable when fractional selection is allowed.

2. Shortest Path Comparison (Dijkstra vs Bellman-Ford)

Given a graph with edges including negative weights, apply both Dijkstra's and Bellman-Ford algorithms to compute shortest paths. Compare their correctness, complexity, and limitations. Critically evaluate why Dijkstra fails in certain cases. Analyze performance differences and derive insights on algorithm selection.

Problem:

Given a graph containing edges with negative weights, apply **Dijkstra's** and **Bellman-Ford** algorithms to compute shortest paths. Compare their correctness, complexity and limitations, and critically evaluate why Dijkstra fails in certain cases.

1. Problem Definition with Suitable Example

The **Shortest Path Problem** determines the minimum-cost path from a source vertex to all other vertices in a weighted graph.

Two commonly used algorithms are **Dijkstra's Algorithm** and **Bellman-Ford Algorithm**.

- **Dijkstra's Algorithm** selects the unvisited vertex with the smallest known distance and updates its neighbouring vertices.
- **Bellman-Ford Algorithm** repeatedly relaxes all edges and can handle negative edge weights.

Consider the following directed graph:

Edge Weight

$0 \rightarrow 1$ 4

$0 \rightarrow 2$ 5

$2 \rightarrow 1$ -3

$1 \rightarrow 3$ 2

$2 \rightarrow 3$ 6

Source vertex = 0

The shortest paths are:

- $0 \rightarrow 2 = 5$
- $0 \rightarrow 2 \rightarrow 1 = 5 - 3 = 2$
- $0 \rightarrow 2 \rightarrow 1 \rightarrow 3 = 2 + 2 = 4$

Therefore, the correct shortest distances are:

This example demonstrates why a negative edge can cause Dijkstra to produce an incorrect result.

2. Algorithm / Pseudocode

Dijkstra's Algorithm

1. Initialize the source distance as 0 and all other distances as infinity.

2. Select the unvisited vertex with the smallest distance.
3. Mark it as visited.
4. Relax all its neighbouring edges.
5. Repeat until all vertices are processed.

Pseudocode:

Set source distance = 0

Set all other distances = infinity

Repeat:

 Select unvisited vertex with minimum distance

 Mark it as visited

 For each neighbour:

 Update distance if a shorter path is found

Return distances

Bellman-Ford Algorithm

1. Initialize the source distance as 0.
2. Set all other distances to infinity.
3. Relax every edge $V-1$ times.
4. Perform one additional relaxation to detect a negative cycle.
5. Return the shortest distances.

Pseudocode:

Set source distance = 0

Set all other distances = infinity

Repeat $V-1$ times:

 For every edge (u,v,w) :

 If $d[u] + w < d[v]$:

 Update $d[v]$

Check once more for negative cycle

Return distances

3. DP Table / State Representation

For shortest-path algorithms, the state is represented by the distance array.

Let:

represent the shortest known distance from the source vertex to vertex .

For the given graph, the Bellman-Ford relaxation gives:

Vertex Initial Final Shortest Distance

0	0	0
1	∞	2
2	∞	5
3	∞	4

Therefore:

Dijkstra may incorrectly finalize vertex 1 with distance **4** before discovering the path:

whose cost is:

4. Recurrence Relation

For both algorithms, the basic relaxation operation is:

where:

- = current distance to vertex
- = weight of edge
- = current distance to vertex

Bellman-Ford

The relaxation is performed for every edge repeatedly:

for **V – 1 iterations**.

Dijkstra

Dijkstra uses the same relaxation idea, but once a minimum-distance vertex is selected and finalized, its distance is assumed to be optimal.

This assumption is valid only when edge weights are non-negative.

5.Implementation

SIMATS
ENGINEERING

SIMATS Online Programming Lab
DAA PROGRAMMING

Soundarya M
B2324345678

Language: Python C C++ Java

QUESTIONS

Q1
Greedy Technique
50 marks

Q2
Greedy Technique
50 marks

2/2 answered

Q2 Greedy Technique 50 marks

Shortest Path Comparison (Dijkstra vs Bellman-Ford)
Given a graph with edges including negative weights, apply both Dijkstra's and Bellman-Ford algorithms to compute shortest paths. Compare their correctness, complexity, and limitations. Critically evaluate why Dijkstra fails in certain cases. Analyze performance differences and derive insights on algorithm selection.

Your Code

```

1 import heapq
2
3 def dijkstra(n, edges, start_node):
4     # Create adjacency list
5     adj = [[] for i in range(n)]
6     for u, v, w in edges:
7         adj[u].append((v, w))
8
9     distances = [float('inf')] * n
10    distances[start_node] = 0
11    pq = [(0, start_node)]
12
13    while pq:
14        current_dist, u = heapq.heappop(pq)
15
16        if current_dist > distances[u]:
17            continue
18
19        for v, weight in adj[u]:
20            distance = current_dist + weight
21            if distance < distances[v]:
22                distances[v] = distance
23                heapq.heappush(pq, (distance, v))
24
25    return distances
26
27 def bellman_ford(n, edges, start_node):
28     distances = [float('inf')] * n
29     distances[start_node] = 0
30
31     # Relax edges n-1 times

```

Input

```

5 6
0 1 4
0 2 2
2 1 1

```

Output

```

Dijkstra Results: [0, 1, 2, 3, 4]
(Note: Dijkstra might give wrong results if negative edges exist!)

```

6. Time Complexity Analysis

Dijkstra

Using a simple array implementation:

Using a priority queue:

Bellman-Ford

Every edge is relaxed $V-1$ times.

Therefore:

Thus, Dijkstra is generally faster than Bellman-Ford when its non-negative-weight condition is satisfied.

7. Space Complexity Analysis

Dijkstra

The distance array, visited array and graph representation require:

for an adjacency-list representation.

Bellman-Ford

The edge list and distance array require:

Therefore, both algorithms can require similar space when the graph is stored using an adjacency list/edge list.

8. Comparison of the Two Approaches

Parameter	Dijkstra	Bellman-Ford
Technique	Greedy	Repeated relaxation

Parameter	Dijkstra	Bellman-Ford
Negative edges	Not supported correctly	Supported
Negative cycles	Cannot handle	Can detect
Time Complexity	$O(V^2)$ / $O((V+E)\log V)$	$O(VE)$
Speed	Faster	Slower
Main operation	Select minimum-distance vertex	Relax all edges
Optimality	Correct for non-negative weights	Correct with negative edges, if no negative cycle
Best use	Non-negative weighted graphs	Graphs containing negative weights

9. Similarities Between the Two Approaches

1. Both are shortest-path algorithms.
 2. Both calculate the shortest distance from a source vertex.
 3. Both use edge relaxation to improve distance estimates.
 4. Both maintain a distance value for each vertex.
 5. Both can be used with weighted graphs.
 6. Both produce correct shortest paths when their respective conditions are satisfied.
-

10. Differences in State Definition and Transitions

In **Dijkstra**, the algorithm maintains a set of finalized vertices. The vertex having the smallest current distance is selected and treated as permanently finalized.

Therefore, its transition is based on:

after selecting the minimum-distance vertex.

In **Bellman-Ford**, there is no permanent finalization of a vertex during the main process. Instead, **all edges are repeatedly relaxed**.

Therefore, Bellman-Ford can discover a better path later, even if the path contains a negative edge.

For the given example:

Initially, Dijkstra considers vertex 1 to have distance 4.

However:

has cost:

Bellman-Ford can discover this improved distance through repeated relaxation.

11. Comparison of State, Transition, Implementation and Complexity

Aspect	Dijkstra	Bellman-Ford
State	Distance + visited/finalized vertices	Distance array
Transition	Relax neighbours of selected minimum vertex	Relax every edge repeatedly
Negative edge	Causes incorrect results	Supported
Implementation	Selection + relaxation	Repeated edge relaxation
Time	$O(V^2)$ or $O((V+E)\log V)$	$O(VE)$
Negative cycle detection	No	Yes
Efficiency	Generally faster	Generally slower
Suitable graph	Non-negative weights	Negative weights allowed

Why Dijkstra fails

Dijkstra assumes that once a vertex has the smallest current distance, no future path can make that distance smaller.

This assumption fails when negative edges are present.

In the example, Dijkstra can finalize:

before processing vertex 2.

But later:

gives:

Therefore, the previously finalized distance of 4 is actually incorrect.

Bellman-Ford does not make this permanent-finalization assumption, so it correctly obtains:

12. Conclusion

Dijkstra and Bellman-Ford both solve the shortest-path problem, but they are suitable for different graph conditions.

Dijkstra's Algorithm is efficient and provides correct results when all edge weights are non-negative. However, it can fail when negative-weight edges are present because its greedy finalization assumption becomes invalid.

Bellman-Ford is slower, with a complexity of $O(VE)$, but it can handle negative edge weights and can also detect negative-weight cycles.

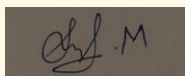
For the given example, the correct shortest distances are:

Hence, Dijkstra should be selected for faster shortest-path computation on graphs with non-negative weights, while Bellman-Ford should be selected when negative edge weights are possible.

Code of Conduct

I Soundariya M (192524342) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:


RUBRICS FOR CALCULATION

Comparative Analysis					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Selection of Studies/Parameters	20%	Selects highly relevant and diverse studies with well-defined comparison parameters	Selects relevant studies with minor gaps in parameter definition	Limited or partially relevant selection	Poor or inappropriate selection of studies
Comparison & Differentiation	25%	Clearly compares multiple aspects (methods, results, limitations) with strong differentiation	Good comparison with minor gaps	Basic comparison with limited depth	No clear comparison; mostly descriptive
Critical Evaluation	25%	Critically evaluates strengths, weaknesses, and implications with strong justification	Provides evaluation with some justification	Limited evaluation; lacks depth	No critical evaluation provided
Synthesis & Insight Generation	20%	Generates meaningful insights and conclusions from comparisons	Some insights derived from comparison	Limited or unclear insights	No insights generated
Clarity	10%	Information is clearly presented (tables/figures) with logical structure	Generally clear with minor issues	Presentation lacks clarity or structure	Poor presentation and difficult to understand

Mark Evaluation:

Questions	Selection of Studies/Parameters (20)	Comparison & Differentiation (25)	Critical Evaluation (25)	Synthesis & Insight Generation (20)	Clarity (10)	Total Marks (Each - 50)
Q1						
Q2						
Overall Total						

Faculty In-charge

Course Coordinator

Head of Department

Name: _____ Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____